

Road sign detection

1. Course Content

Start the YOLO traffic sign recognition program. The model provided in the tutorial only recognizes the signs included in the sand table package. If you need to recognize other signs, you need to train the model yourself.

2. Sign Detection

Sign detection source code path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

3. Program Startup

- Start the camera node

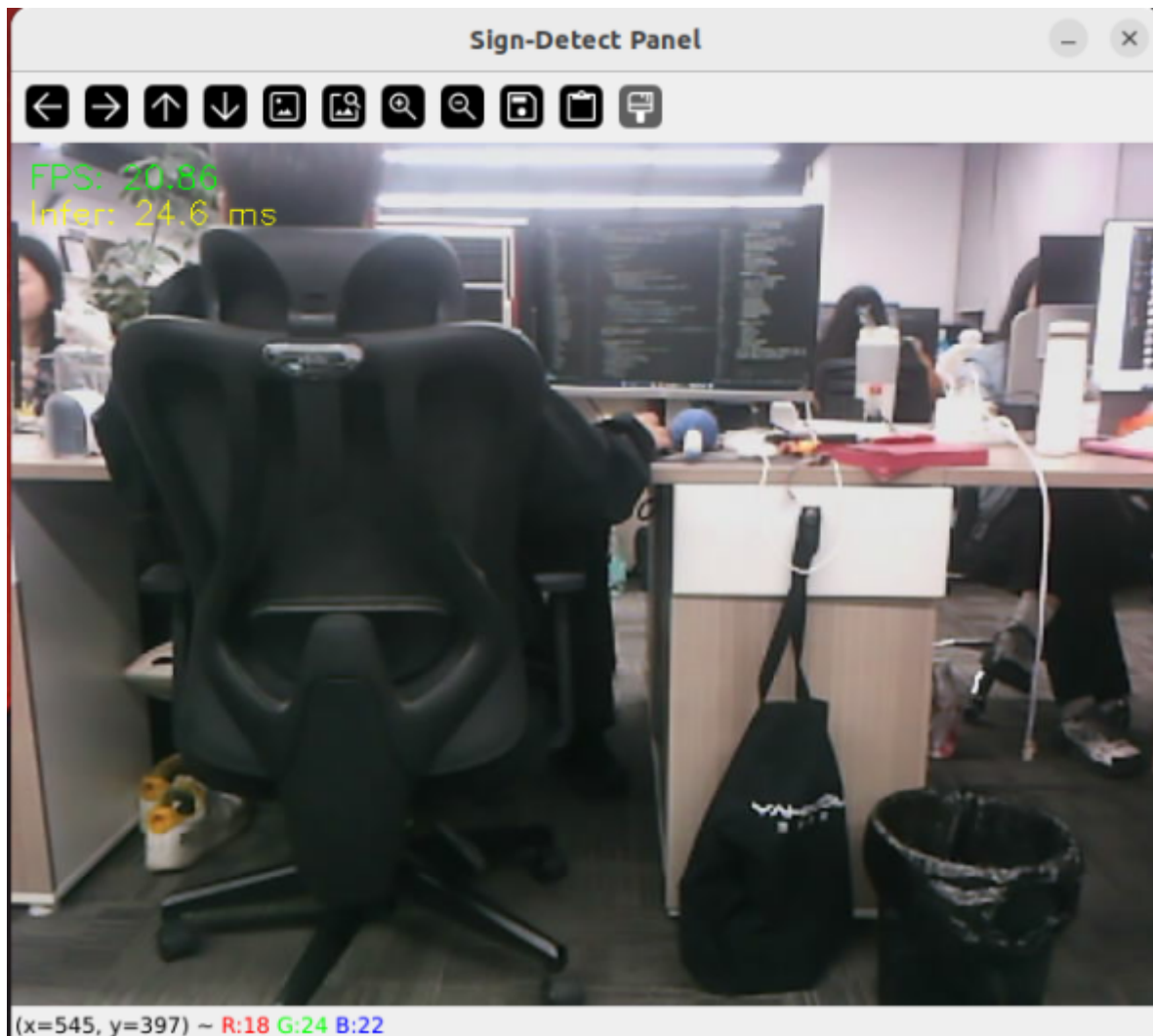
```
ros2 launch ascamera hp60c.launch.py
```

- Start the sign recognition program

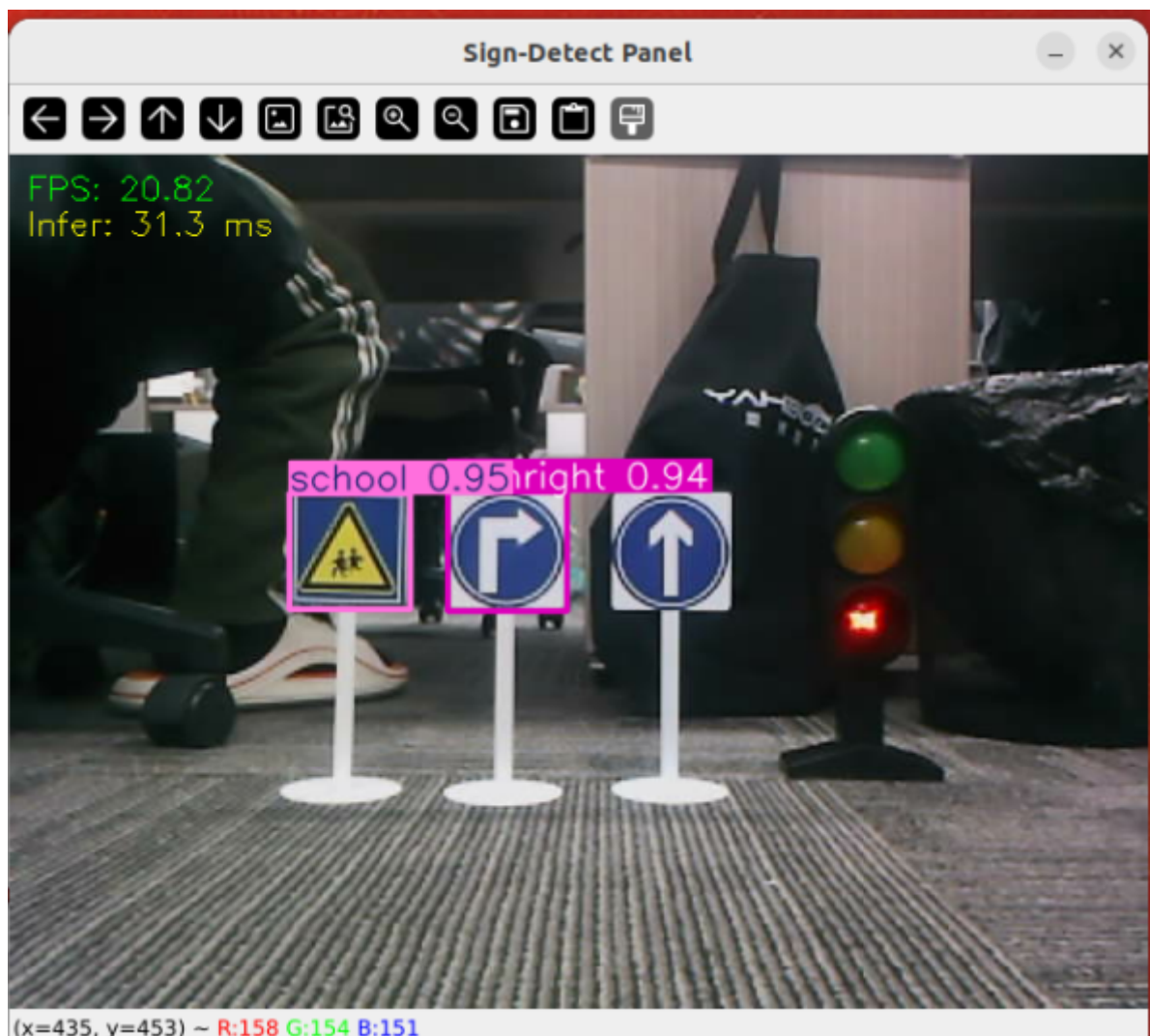
```
ros2 run auto_drive yolo_detect
```

```
jetson@yahboom:~$ ros2 run auto_drive yolo_detect
pygame 2.6.1 (SDL 2.28.4, Python 3.10.12)
Hello from the pygame community. https://www.pygame.org/contribute.html
[INFO] [1764583729.314075916] [YOLODetection]: yolo_detect node started.
Loading /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/sign_model.engine for TensorRT inference...
[12/01/2025-18:08:52] [TRT] [I] Loaded engine size: 12 MiB
[12/01/2025-18:08:52] [TRT] [W] Using an engine plan file across different models of devices is not recommended and is
likely to affect performance or even cause errors.
[12/01/2025-18:08:52] [TRT] [I] [MemUsageChange] TensorRT-managed allocation in IExecutionContext creation: CPU +0, GP
U +19, now: CPU 0, GPU 28 (MiB)
```

After successful startup, a visual interface will appear.



Placing a road sign in the center of the screen will correctly identify its name. The factory default is to recognize only two road signs at a time to prevent misidentification during autonomous driving.



The YOLO recognition performance can be adjusted by modifying the YAML file in this path:

`/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/yolo_param.yaml`

```
#路标检测推理参数
sign_detection:
  model_path: "/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/sign_model.engine"
  conf: 0.92
  iou: 0.7
  rect: True
  half: True
  device: "cuda:0"
  batch: 50
  max_det: 2
  augment: False
  verbose: False
  classes: [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]

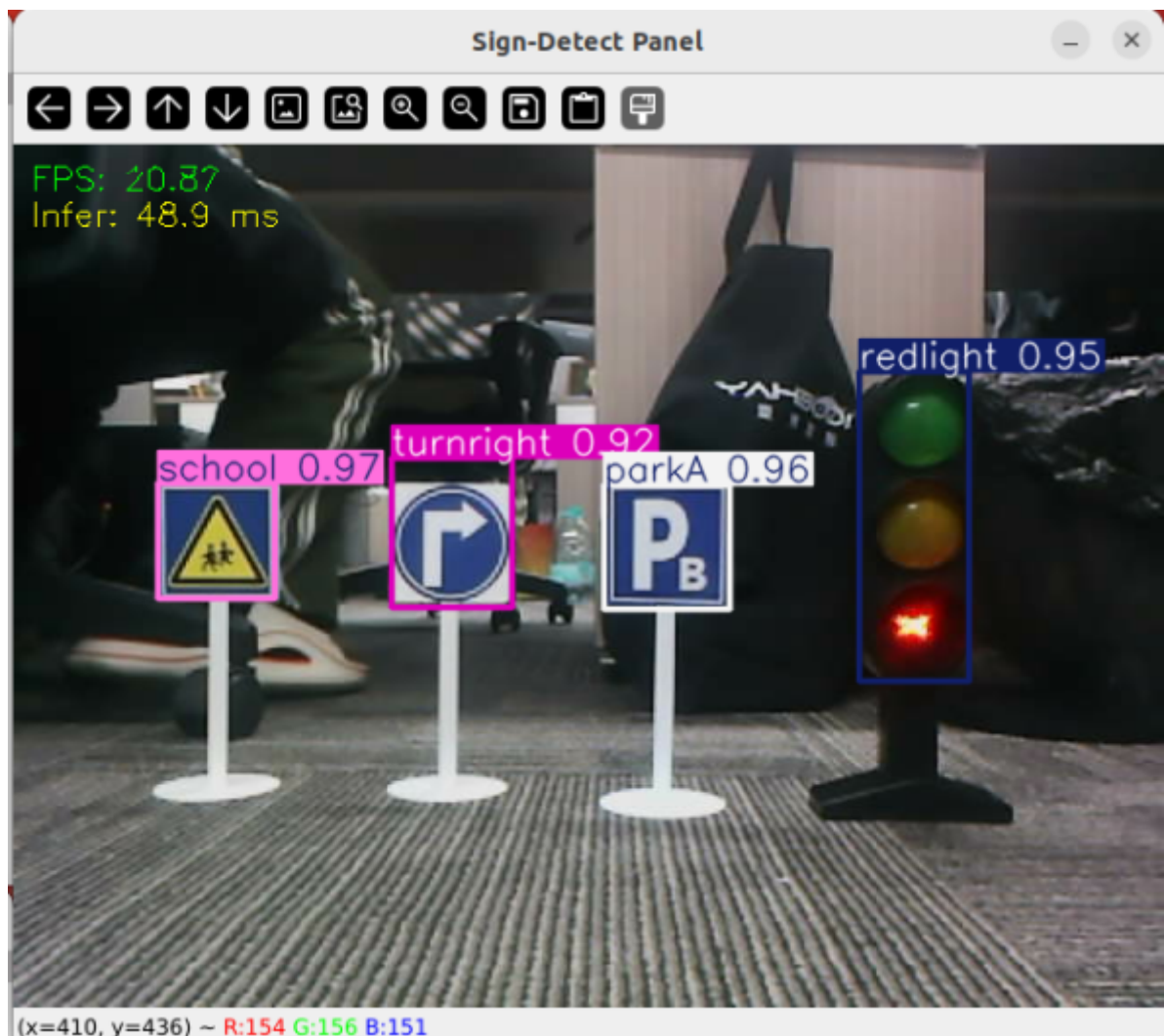
#设置检测的最小置信度阈值。 将忽略置信度低于此阈值的检测到的对象。 调整此值有助于减少误报。
#用于非极大值抑制 (NMS) 的重叠阈值。 较低的值会通过消除重叠的框来减少检测结果。
#如果启用, 则对图像较短的一边进行最小填充, 直到可以被步长整除, 以提高推理速度。
#如果启用, 则使用半精度推理, 可以加快在支持的 GPU 上的模型推理速度, 同时对准确性的影响极小。
#推理设备
#批处理大小, 更大的批处理大小可以提供更高的吞吐量, 从而缩短推理所需的总时间。
#最多检测的物体数
#启用测试时增强 (TTA) 进行预测, 可能会提高检测的鲁棒性, 但会降低推理速度。
#是否在终端输出推理的详细信息。

line_detection:
  #车道线检测推理参数
  model_path: "/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/line_V6.engine"
  conf: 0.5
  iou: 0.6
  rect: True
  half: True
  device: "cuda:0"
  batch: 50
  max_det: 4
  augment: False
  verbose: False
  #设置检测的最小置信度阈值。 将忽略置信度低于此阈值的检测到的对象。 调整此值有助于减少误报。
  #用于非极大值抑制 (NMS) 的重叠阈值。 较低的值会通过消除重叠的框来减少检测结果。
  #如果启用, 则对图像较短的一边进行最小填充, 直到可以被步长整除, 以提高推理速度。
  #如果启用, 则使用半精度推理, 可以加快在支持的 GPU 上的模型推理速度, 同时对准确性的影响极小。
  #推理设备
  #批处理大小, 更大的批处理大小可以提供更高的吞吐量, 从而缩短推理所需的总时间。
  #最多检测的物体数
  #启用测试时增强 (TTA) 进行预测, 可能会提高检测的鲁棒性, 但会降低推理速度。
  #是否在终端输出推理的详细信息。
```

Where `max_det: 2` represents the maximum number of objects to detect, and `conf: 0.92` sets the minimum confidence threshold for detection. Objects detected with a confidence level below this threshold will be ignored. Adjusting this value helps reduce false positives.

After changing `max_det` to 4 and recompiling and running the program, you can select four road signs on the screen.

```
cd yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select auto_drive
```



For future integrated applications in autonomous driving, it is recommended to use two maximum targets.

4. Source Code Analysis

4.1 yolo_detect.py

Program source code path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

A YOLO object detection node based on ROS2, specifically designed for real-time detection of traffic signs. Nodes receive camera images, perform object detection using the YOLO model, and publish the detection results to other nodes.

Multi-threaded Processing Architecture

- **Processing Thread:** Dedicated to YOLO inference and result processing
- **Daemon Thread:** Ensures proper thread termination upon program exit
- **Ignore List:** Filters out flag types that do not require processing

```
# Create a dedicated processing thread
handle_sign_thread = threading.Thread(target=self.handle_sign)
handle_sign_thread.daemon = True # Set as a daemon thread
handle_sign_thread.start()

# Ignore flag list
self.ignored_signs = ["turnright_ground", "sidewalk_ground"]`
```

Image Callback Function

- Use `cv_bridge` to convert ROS image messages to OpenCV format.
- Create image copies to avoid data races.
- Place images into a processing queue for use by the detection thread.

```
def image_callback(self, msg: Image):
    '''ROS image callback function adds the image to the processing queue.'''
    cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    self.sign_detect.put_image(cv_image.copy())
```

Object detection main loop

```
def handle_sign(self):
    '''Processing road sign detection results in images'''
    while True: # Infinite loop, continuous processing
        try:
            # Retrieve images from the queue (non-blocking method)
            frame = self.sign_detect.image_queue.get(timeout=0.01)
        except queue.Empty:
            continue # Continue looping when the queue is empty.

        # Record the start time of reasoning
        infer_start = time.time()

        # Perform YOLO inference
        results = self.sign_detect.get_infer_result(frame, True)
```

Result Parsing and Publishing

- **Frequency Control:** Publish every 5 frames to balance real-time performance and system load.
- **Data Type Conversion:** Ensure data format conforms to message type requirements.

```
for res in results:
    if res.bboxes is not None: # Ensure test results are available
        # Draw a labeled image (for display).
        annotated_frame = res.plot()
        boxes = res.bboxes

        # Traverse all detected bounding boxes
        for i in range(len(boxes)):
            # Extract bounding box coordinates (XYXY format)
            box_coords = boxes.xyxy[i].tolist()

            # Get category name and confidence level
            class_name = res.names[int(boxes.cls[i].item())]
```

```

confidence = boxes.conf[i].item()

# Ignore flags
if class_name in self.ignored_signs:
    continue # Skip processing for res in results:
    if res.bboxes is not None:
        annotated_frame = res.plot()
        boxes = res.bboxes
        for i in range(len(boxes)):
            box_coords = boxes.xyxy[i].tolist()# Get the bounding
box coordinates (xyxy format: xmin, ymin, xmax, ymax)
            class_name = res.names[int(boxes.cls[i].item())]# Get
object name

            confidence = boxes.conf[i].item()# Obtain confidence

# Check if it is a flag that needs to be ignored.
if class_name in self.ignored_signs:
    # rospy.loginfo(f"Ignoring sign: {class_name}")
    continue # Skip the ignore flag

# Release of test results
if self.sign_pub_counter % 5 == 0:
    self.sign_publisher.publish(DetectionObject(
        class_name=class_name,
        confidence=confidence,
        xmin=float(box_coords[0]),
        ymin=float(box_coords[1]),
        xmax=float(box_coords[2]),
        ymax=float(box_coords[3])
    ))
self.sign_pub_counter += 1

```